Switch :

A switch statement in JavaScript is a control flow statement used to perform different actions based on different conditions. It evaluates an expression and then executes the code block associated with the first matching case value.

Syntax and Components

The structure of a switch statement consists of the following parts:

1. **switch (expression)** – This is the value or condition that will be evaluated and compared with each case. It is typically a variable or an expression.

2. **case value** – Each case specifies a possible value to match against the expression. When a match is found, the code block associated with that case is executed.

3. **break** – A key statement that terminates the switch once the matching case has been executed. If omitted, the program continues executing the next case statements.
4. **default** – *(Optional)* This block runs when none of the defined case values match the expression. It functions similarly to the else block in an if–else statement.

```
let dayNumber = 5;
let dayName;

switch (dayNumber) {
  case 1:
    dayName = "Monday";
    break;
  case 2:
    dayName = "Tuesday";
    break;
  case 3:
    dayName = "Wednesday";
    break;
  case 4:
    dayName = "Thursday";
    break;
  case 5:
    dayName = "Friday";
    break;
  case 6:
```

```
    dayName = "Saturday";
    break;
  case 7:
    dayName = "Sunday";
    break;
  default:
    dayName = "Invalid day number";
}

console.log(`Today is ${dayName}.`);
```

Loops :

A loop in JavaScript is a programming structure used to execute a block of code repeatedly as long as a specific condition remains true.
Loops play an essential role in handling repetitive tasks efficiently, eliminating the need to write the same code multiple times.
JavaScript offers various types of loops, each designed for different use cases and scenarios.

1. For loop

The for loop is one of the most commonly used and concise looping structures in JavaScript. It is best suited for situations where the number of iterations is known in advance.

```
for (let i = 0; i < 5; i++) {
  console.log("Iteration number: " + i);
}
// Output: Iteration number: 0, 1, 2, 3, 4
```

2. While loop

The while loop repeatedly executes a block of code as long as a given condition evaluates to true. The condition is tested before the loop body is executed.

```
let counter = 0;
while (counter < 3) {
  console.log("counter is: " + counter);
  counter++;
```

```
}
// Output: counter is: 0, 1, 2
```

3. do...while Loop

The do...while loop works like a while loop, but the code block is guaranteed to execute at least once, since the condition is evaluated after the first iteration.

```
let x = 10;
do {
  console.log("This runs once even though x > 5: " + x);
  x++;
} while (x < 5);
// Output: This runs once even though x > 5: 10
```

4. for...of Loop

The for...of loop is used to traverse the values of iterable objects, such as arrays, strings, maps, sets, and other iterable structures.

```
const colors = ["red", "green", "blue"];
for (const color of colors) {
  console.log(color);
}

const word = "HELLO";
for (const letter of word) {
  console.log(letter);
}
```

5. for...in Loop

This loop is used to traverse the keys (property names) of an object.

Note that it is generally not recommended for arrays, since it may also iterate over inherited properties.

```
const car = {
  brand: "Toyota",
  model: "Corolla",
  year: 2020
};

for (const key in car) {
  console.log(`${key}: ${car[key]}`);
}
```

Type Conversion

Type conversion is the process of converting a value from one data type to another. JavaScript performs type conversion in two ways:

1. **Implicit Conversion (Type Coercion)** :
2. **Explicit Conversion (Type Casting)** :

Implicit Conversion:

JavaScript automatically converts types when needed.

```
let result = "Hello" + 5;
console.log(result); // "Hello5"
console.log(typeof result); // "string"

let result1 = 10 - "4";
console.log(result1); // 6
console.log(typeof result1); // "number"

console.log(true + 3);  // 4 (true → 1)
console.log(false + 5); // 5 (false → 0)

let result2 = "Value is " + true;
```

```
console.log(result2); // "Value is true"
console.log(typeof result2); // "string"

console.log(null + 5);      // 5 (null → 0)
console.log(undefined + 5);  // NaN (undefined → NaN)

console.log(5 == "5");   // true (string "5" → number 5)
console.log(false == 0); // true (false → 0)
console.log(null == 0);  // false (null only equals undefined)
```

Explicit Conversion:

You manually convert a value using functions or operators.

```
//String to Number
let strNumber1 = "123";
let num1 = Number(strNumber1);
console.log(num1, typeof num1);  // 123 "number"

let strNumber2 = "45.67";
let num2 = parseFloat(strNumber2);
console.log(num2, typeof num2);  // 45.67 "number"

//Number to String
let number1 = 100;
let str1 = String(number1);
console.log(str1, typeof str1);  // "100" "string"

let number2 = 250;
let str2 = number2 + "";
console.log(str2, typeof str2);  // "250" "string"

//Boolean to Number
let bool1 = true;
let num3 = Number(bool1);
console.log(num3, typeof num3); // 1 "number"

let bool2 = false;
let num4 = Number(bool2);
console.log(num4, typeof num4); // 0 "number"
```

```
//Number to Boolean
let num5 = 10;
let bool3 = Boolean(num5);
console.log(bool3, typeof bool3); // true "boolean"

let num6 = 0;
let bool4 = Boolean(num6);
console.log(bool4, typeof bool4); // false "boolean"


//String to Boolean
let strBool1 = "Hello";
let bool5 = Boolean(strBool1);
console.log(bool5, typeof bool5); // true "boolean"

let strBool2 = "";
let bool6 = Boolean(strBool2);
console.log(bool6, typeof bool6); // false "boolean"


//Boolean to String
let bool7 = true;
let str3 = String(bool7);
console.log(str3, typeof str3); // "true" "string"

let bool8 = false;
let str4 = String(bool8);
console.log(str4, typeof str4); // "false" "string"
```

Error :

A JavaScript error indicates that something went wrong during the execution of a script. These errors can stop your code from running or cause unexpected behavior.

Types of JavaScript Errors :

a) Syntax Errors

- Occur when the code structure is incorrect.
- These errors are detected before execution.

```
console.log("Welcome to JavaScript"  // Missing closing parenthesis
```

b) Reference Errors

- Happen when you try to access a variable or function that doesn't exist or isn't declared yet.

```
console.log(userName); // userName is not defined
```

c) Type Errors

- Occur when an operation is performed on a value of an inappropriate type.

```
let number = 42;
number.toLowerCase(); // Attempting string method on number
```

d) Range Errors

- Happen when a value is outside the allowed range.

```
let arr = new Array(1000000000000); // Invalid array size
```

e) URI Errors

- Occur when URI functions are used incorrectly.

```
decodeURIComponent("%ZZ"); // Invalid URI sequence
```

Error Handling :

Error handling in JavaScript is the process of detecting and responding to runtime errors (unexpected issues that occur during program execution).
JavaScript provides built-in mechanisms like `try...catch...finally` and custom `throw` statements to manage these errors gracefully — preventing the program from crashing.

Block	Description
try	Contains the code that might throw an error.
catch	Executes when an error occurs inside the try block. The `error` object provides details about what went wrong.
finally	Always executes, regardless of whether an error occurred — useful for cleanup tasks (like closing connections).

```
try {
  console.log("Before error");
  console.log(10 / 0);  // Valid operation, outputs Infinity
  console.log(x);       //  x is not defined
}
catch (error) {
  console.log("An error occurred: " + error.message);
}
finally {
  console.log("Execution completed.");
}
```

Debugging :

Debugging in JavaScript is the process of finding and fixing errors or bugs in your code. It helps developers understand why their program isn't behaving as expected and corrects mistakes to ensure smooth execution.

Why Debugging Is Important

- Helps identify and fix logical or runtime errors.
- Improves code quality and reliability.
- Saves time during development by isolating problem areas quickly.

Some Common term used in Debugging :

Breakpoint: A marker that pauses code execution at a specific line so you can inspect the program's state.

Step Over: Executes the current line of code and moves to the next one, skipping over any called functions.

Step Into: Enters inside a called function to debug it line by line.

Step Out: Finishes executing the current function and returns to the line that called it.

Watch Variable: Lets you monitor the value of a specific variable as the program runs or pauses.

Call Stack: Displays the sequence of function calls that led to the current point in the program.

DOM Breakpoint: Pauses execution when a change occurs to a selected HTML element (like modification, removal, or attribute change) in the DOM.

1. Using console.log()

The simplest way to debug is by printing values to the console.

```
const price = 100;
const discount = 20;
console.log("Price before discount:", price);
```

```
console.log("Discount applied:", discount);
console.log("Final price:", price - discount);
```

2. Using console.error() and console.warn()

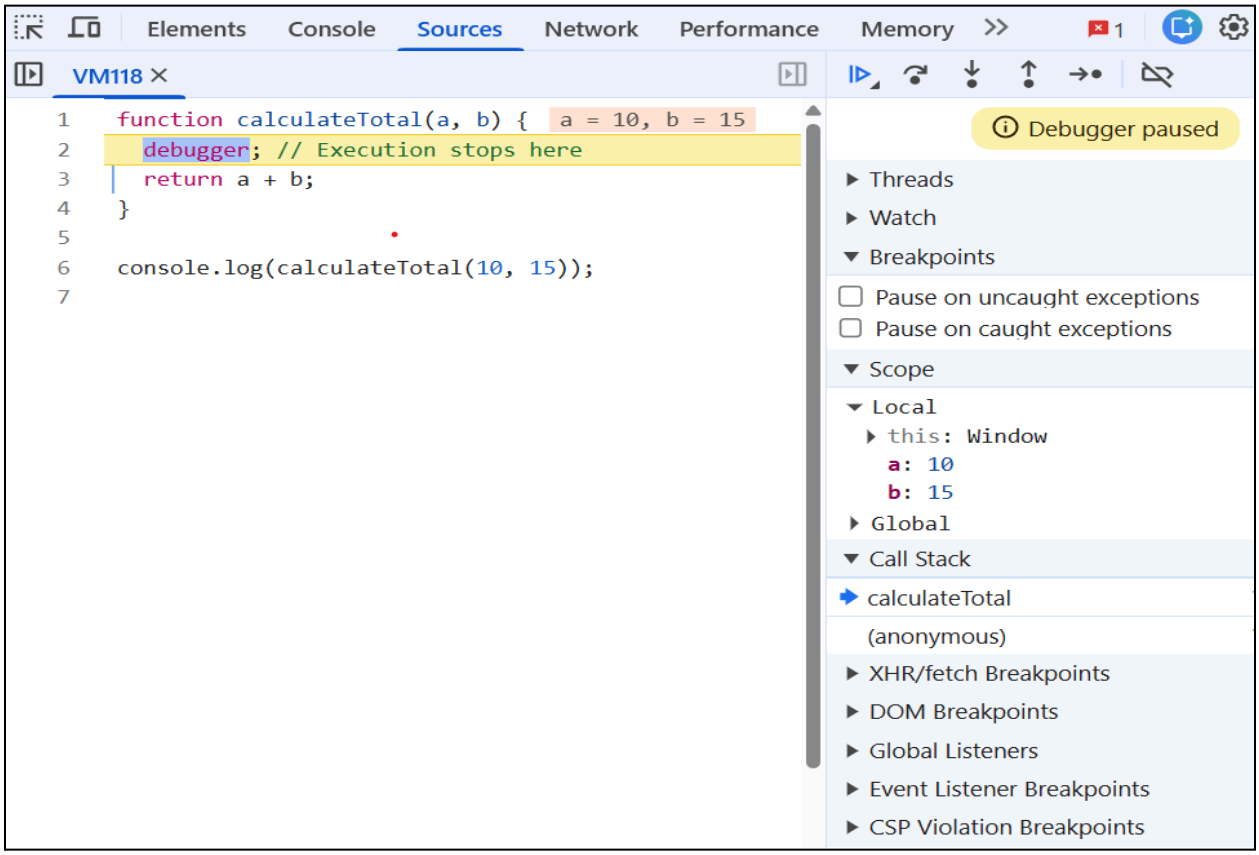
- console.error() highlights errors in the console.
- console.warn() shows warnings.

```
let age = -5;

if(age < 0) {
  console.warn("Warning: Age cannot be negative");
}

try {
  let num = "hello";
  console.log(num.toFixed(2));
} catch(err) {
  console.error("Error occurred:", err.message);
}
```

3. Using the debugger Keyword

The debugger statement pauses code execution, allowing you to inspect variables and step through code line by line in browser DevTools or VS Code.

```
function calculateTotal(a, b) {
  debugger; // Execution stops here
  return a + b;
}
console.log(calculateTotal(10, 15));
```



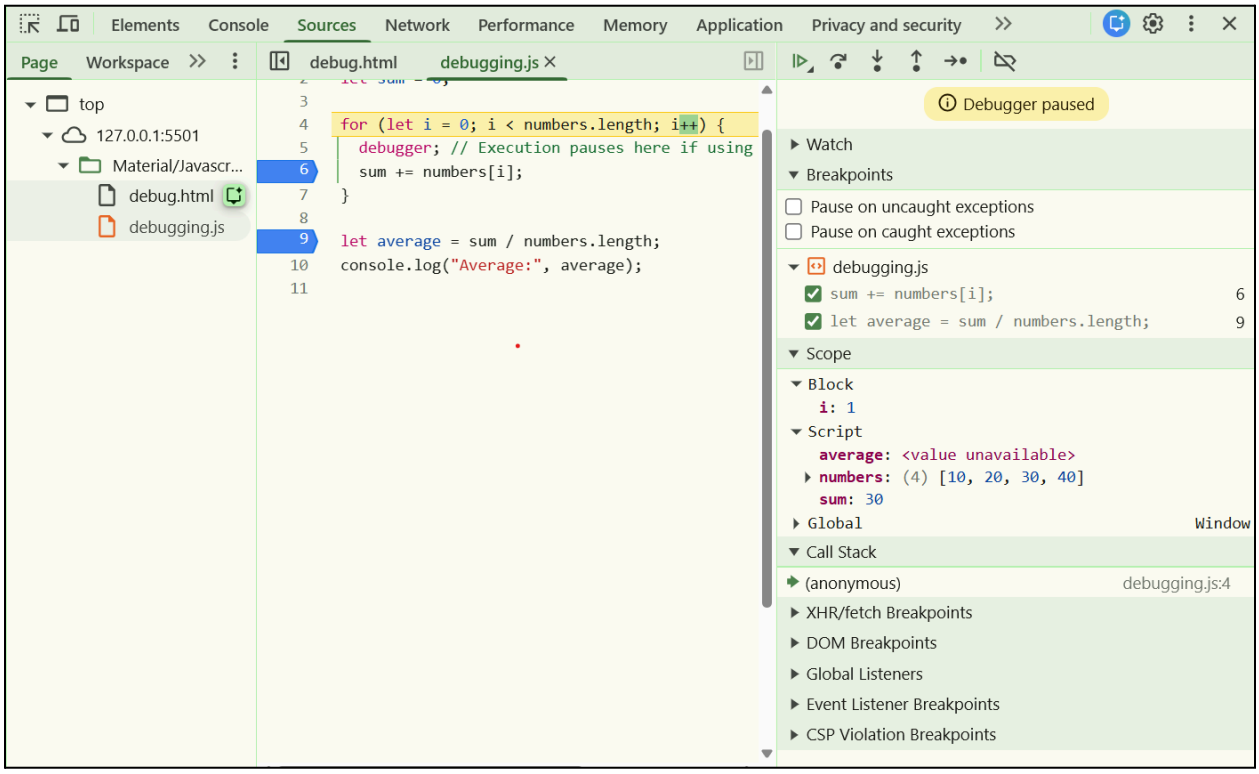
4. Using Browser Developer Tools

- Open DevTools in Chrome (press F12 or Ctrl + Shift + I).
- Go to the Sources tab to set breakpoints.
- Use Step Over, Step Into, and Watch Variables to analyze the code flow.

Example :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <script src="./debugging.js"></script>
</head>
<body>
  <h1>Debugging</h1>
</body>
</html>
```

```
const numbers = [10, 20, 30, 40];
let sum = 0;

for (let i = 0; i < numbers.length; i++) {
  debugger; // Execution pauses here if using browser DevTools or VS Code
  sum += numbers[i];
}

let average = sum / numbers.length;
console.log("Average:", average);
```



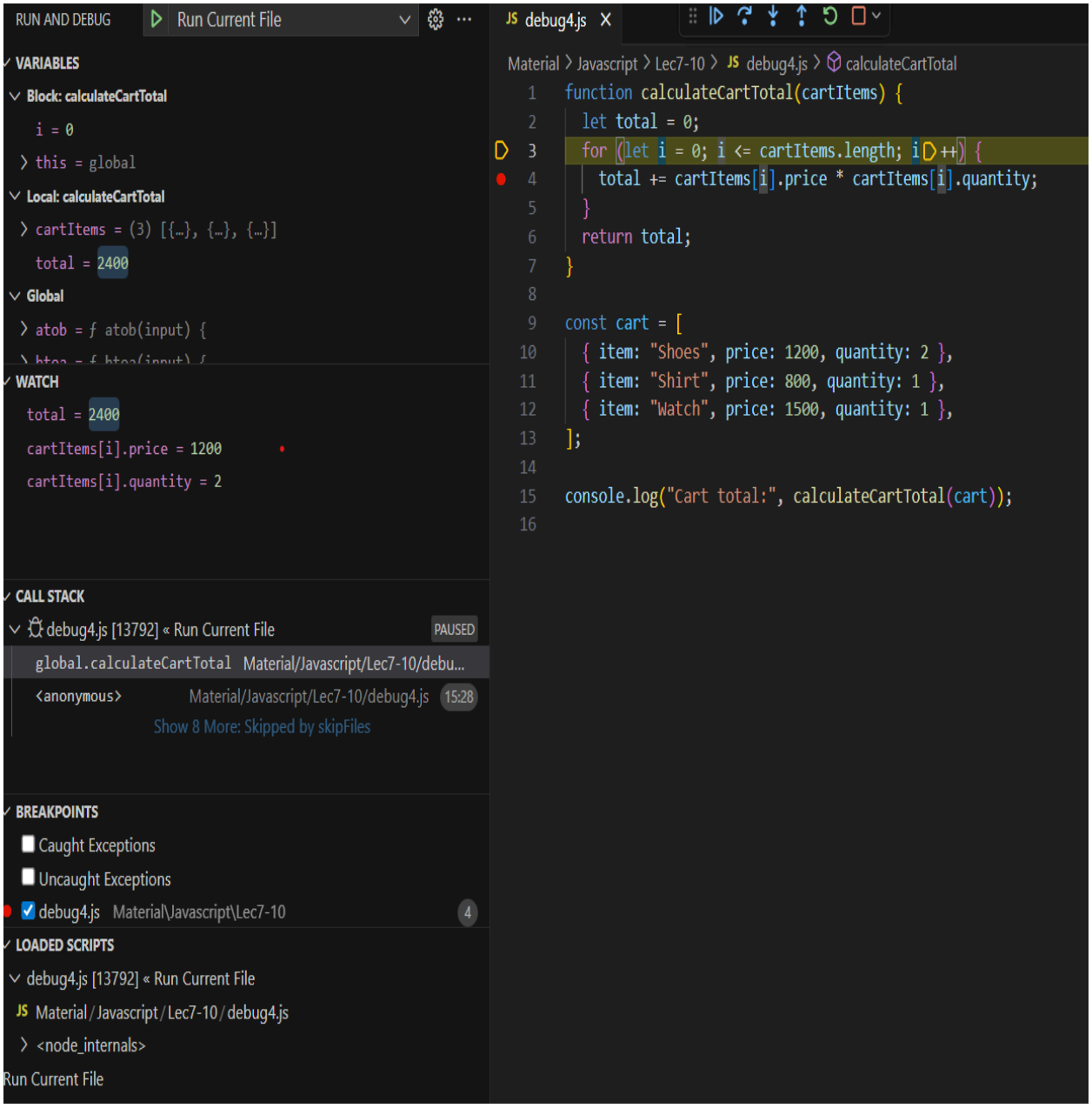
5. **VS Code Debugger**

- VS Code provides a built-in **JavaScript debugger**.
- Steps:
    1. Open your JavaScript file in VS Code.
    2. Press **F5** to start debugging.
    3. Set **breakpoints** by clicking beside the line numbers.

4. Inspect variables and watch expressions.

Example :

```
function calculateCartTotal(cartItems) {
  let total = 0;
  for (let i = 0; i <= cartItems.length; i++) {
    total += cartItems[i].price * cartItems[i].quantity;
  }
  return total;
}

const cart = [
  { item: "Shoes", price: 1200, quantity: 2 },
  { item: "Shirt", price: 800, quantity: 1 },
  { item: "Watch", price: 1500, quantity: 1 },
];

console.log("Cart total:", calculateCartTotal(cart));
```



Hoisting

Hoisting refers to the behavior where JavaScript moves the declarations of variables, functions, and classes to the top of their scope during the compilation phase, before the code is executed.

When a script runs, the JavaScript engine scans the code first, registers all variable and function declarations, and then executes the code line by line.

The Internal Working of Hoisting :

When JavaScript runs your code, two main phases occur:

1. **Memory Creation Phase**
    a. All variables and functions are allocated in memory.
    b. Variables declared with var are initialized with undefined.
    c. Function declarations are stored completely in memory.
2. **Execution Phase**
    a. The code is executed line by line.
    b. Variable assignments and other operations take place in this phase.

There are mainly three types of hoisting

1. Function Hoisting

a. Function Declaration :

```
greet();
function greet() {
  console.log("Hello World!");
}
```

Function declarations are fully hoisted, meaning their complete definition is available before execution begins.

b. Function Expression:

```
sayHi();  //TypeError
var sayHi = function() {
  console.log("Hi there!");
};
```

```

```

The variable sayHi is hoisted and initialized as undefined.

The function body is assigned later, so calling it before initialization throws TypeError: sayHi is not a function.

2. Variable Hoisting

Hoisting with var :

```
console.log(a); // Output: undefined
var a = 10;
console.log(a); // Output: 10
```

During the memory phase, var a is hoisted and initialized as undefined.

The actual value 10 is assigned later during execution.

Hoisting with let and const:

Variables declared with let and const are also hoisted, but they are not initialized. If you try to access them before the actual declaration line in the code, JavaScript throws a ReferenceError.

The area from the start of the scope until the line where let/const is declared is called the Temporal Dead Zone (TDZ).

```
console.log(x); //  ReferenceError
let x = 5;

console.log(y); //  ReferenceError
const y = 10;
```

3. Class Hoisting

```

```

```
const student1 = new Student("Alice", 22); // ReferenceError

class Student {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  displayInfo() {
    console.log(`Name: ${this.name}, Age: ${this.age}`);
  }
}

const student2 = new Student("Bob", 20); // Works fine
student2.displayInfo();
```

Even though class declarations are hoisted, they are not initialized until the code reaches their declaration line.

This means the class is in a Temporal Dead Zone (TDZ) — just like variables declared with let or const.

Trying to access or instantiate the class before its declaration throws a ReferenceError.

Once the declaration is reached, the class becomes available for use.

Example of hoisting :

```
// Trying to access before declaration
console.log(a);         // undefined (var is hoisted)
greet();                // Works (function declaration is hoisted)
try {
  console.log(Person);  //  ReferenceError (class is hoisted but in TDZ)
} catch (e) {
  console.log(e.message);
}

// Variable declared using var
var a = 10;

// Function declaration
```

```
function greet() {
  console.log("Hello from the greet function!");
}

// Arrow function assigned to var (acts as expression)
try {
  sayHi(); // TypeError: sayHi is not a function
} catch (e) {
  console.log(e.message);
}

var sayHi = () => {
  console.log("Hi there!");
};

// Class declaration
class Person {
  constructor(name) {
    this.name = name;
  }

  display() {
    console.log(`Person name: ${this.name}`);
  }
}

// Access after declaration
console.log(a); // 10
sayHi(); // Works now
const p1 = new Person("Alice");
p1.display();
```

Explanation :

1. Memory Creation (Hoisting) Phase

Before the code executes, JavaScript scans the file and sets up memory:

1. var a → created and initialized as undefined.
2. function greet() → entire function body stored in memory (fully hoisted).
3. var sayHi → created and initialized as undefined.
4. class Person → hoisted but uninitialized (in TDZ until declaration).

2. Execution Phase :

Line	Code	Behavior
1	console.log(a)	Outputs undefined — var a is hoisted, but not yet assigned
2	greet()	Works fine — Function declaration is fully hoisted
3	console.log(Person)	Throws ReferenceError — Class is in the Temporal Dead Zone (TDZ)
4	var a = 10;	Assigns value — now a holds 10
5	sayHi() (before declaration)	TypeError — sayHi is undefined at this point
6	var sayHi = () => { ... }	Assigns function — sayHi is now usable
7	class Person { ... }	Declared — Person class becomes accessible
8	console.log(a)	Outputs 10 — uses updated value

9	sayHi()	Works — function now defined
10	new Person("Alice")	Works — class is now available

Strict Mode :

Strict mode in JavaScript is a feature that allows you to write JavaScript code in a restricted variant, effectively opting into a safer and cleaner subset of the language.

It eliminates many "silent errors" by changing them into explicit errors, fixes mistakes that make it difficult for JavaScript engines to perform optimizations, and prohibits some syntax likely to be defined in future versions of ECMAScript.

You activate strict mode by placing the string literal "use strict"; at the beginning of a script or a function.

1. Global Strict Mode (Entire Script)

Placing it at the very top of a JavaScript file applies strict mode to all the code within that file.

```
"use strict";
x = 10; // Error: x is not declared
console.log(x);
```

2. Function-Level Strict Mode (Specific Function)

Placing it inside a function body applies strict mode only to that function and its nested code. The rest of the script remains in non-strict (sloppy) mode. This is useful for migrating large, older codebases gradually.

```
function test() {
  "use strict";
  y = 20; // Error: y is not declared
  console.log(y);
}
test();
```

Strict Mode Alters Normal JavaScript Behavior

Behavior Change	Non-Strict Mode	Strict Mode	Explanation
Implicit Global Variables	Allowed	**Error**	In normal JS, assigning to an undeclared variable creates a global variable automatically. In strict mode, it throws an error. x = 10; // Error in strict mode
Duplicate Function Parameters	Allowed	**SyntaxError**	You can't define two parameters with the same name. function add(a, a) { return a; } // Error
this in Functions	Defaults to global object (window in browsers)	undefined	Prevents accidental modification of the global scope. function show() { console.log(this); } show(); // undefined

Deleting Variables or Functions	Allowed (ignored silently)	**Error**	You cannot delete declared variables or functions. var x = 5; delete x; // Error
Assigning to Read-Only Properties	Ignored silently	**TypeError**	Trying to modify read-only properties now throws an error. "use strict"; const obj = {}; Object.defineProperty(obj, "id", {value: 10, writable: false}); obj.id = 20; // Error
Octal Literals (like 012)	Allowed	**SyntaxError**	Strict mode disallows octal numbers. "use strict"; let num = 010; // Error
with Statement	Allowed	**Error**	The with statement is not permitted in strict mode. "use strict"; with (Math) { console.log(sqrt(16)); } // Error
Eval Scope	Variables inside eval leak outside	**Isolated**	Variables declared in eval() stay local to it. "use strict"; eval("var x = 10;"); console.log(x); // ReferenceError
Future Reserved Words	Can be used as identifiers	**Error**	Cannot use implements, interface, private, etc., as variable names.

Silent Failures Become Errors	Fails silently	**Throws Errors**	Strict mode surfaces silent problems for better debugging.

Benefits of Using Strict Mode

1. Helps avoid accidental globals.
2. Makes debugging easier by throwing clearer errors.
3. Enables safer JavaScript for future updates.
4. Improves performance in modern JavaScript engines.